

# Visual Transform Changes Debugger

This tool will enable you to view and debug changes to position/rotation/scale on **any** transform done by **any** method/setter on a visual frame by frame basis.

It'll will work with code that executes changes via property setters (eg `Transform.position = newValue`) but also more complex methods (eg `Transform.Rotate(1f, 230f, 15f, Space.Self)`).

Changes can be originating from **any** assembly - not only your code but also 3rd party libraries or Unity assemblies.

There are 4 core features that this documentation goes into more detail:

1) View changes to position/rotation/scale of any transform in easy to use GUI on a frame-by-frame basis ([more details](#)) 2) Identify and view specific `TransformModifiers` (specific game object instance and script/method combination that initiated change (eg. Mover-1-GO via TestMove.cs script, method `MoveTransform()`) ([more details](#)) 3) Enable you to easily skip changes originating from specific `TransformModifiers` (at runtime, without need to recompile) - this means you can identify offending code by isolating specific changes ([more details](#)) 4) Reply/preview any frame changes directly in scene-view with a simple button click, making it's easier to pinpoint exact change causing the issue ([more details](#))

**The best place to start are demos available from tool welcome screen. Those were build to get you introduced to tool workflow in simplest way.**

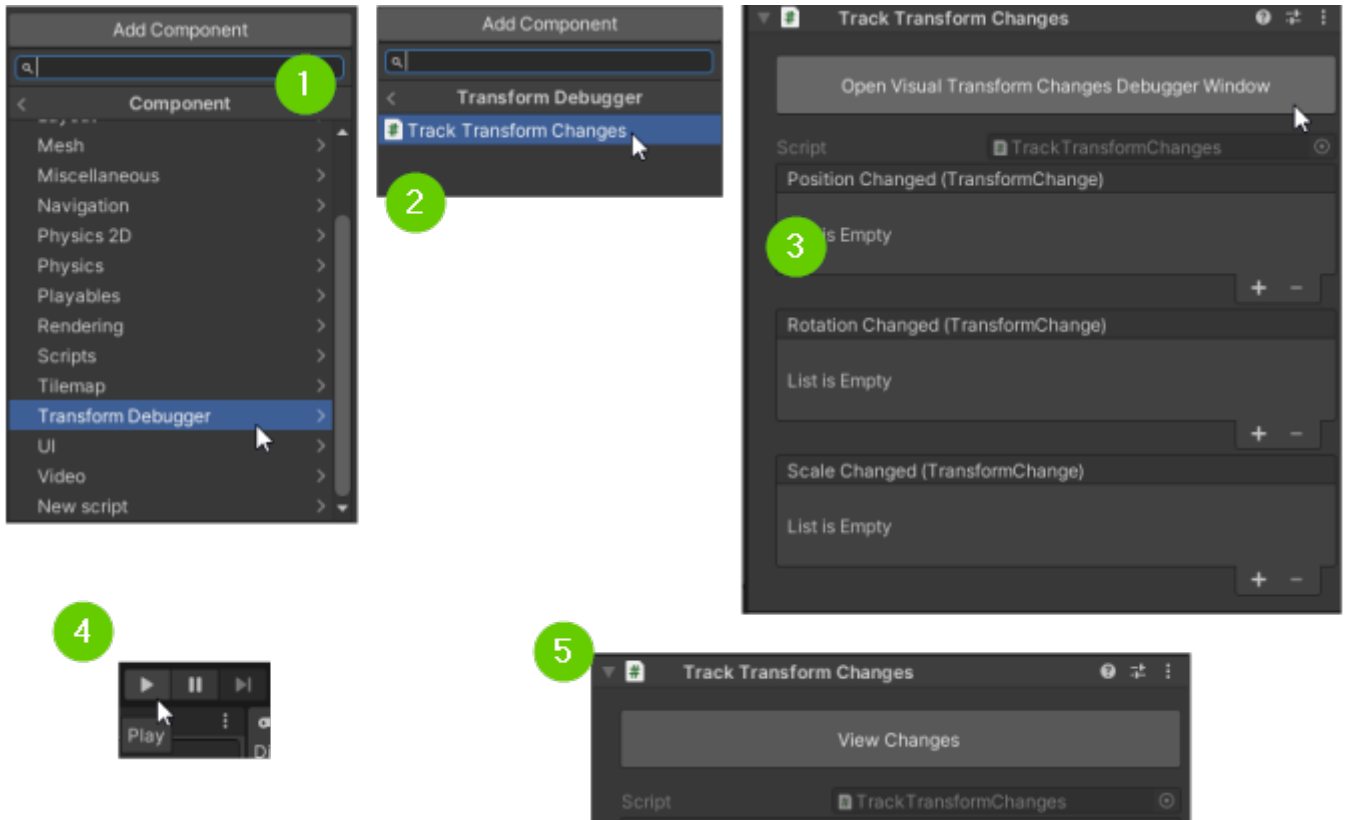
## Table of contents

1. [Setup / Quickstart](#)
2. [Debugger Window](#)
3. [Tracked Objects Sidebar](#)
4. [Modifiers Sidebar](#)
5. [Temporarily Skipping Specific Changes at Runtime](#)
6. [Performance](#)
7. [API Usage - Advanced](#)
8. [API Reference](#)

## Quickstart

Setup is incredibly simple:

1) Import asset into solution 2) Add Track Transform Changes to a component you'd like to track 3) Open tool window 4) Play 5) Hit **View Changes**



## Debugger Window

That's all the setup needed, from now on changes will be recorded in play mode and you'll be able to review them in the debugger window.

## Tracked Objects ?

Track currently selected object  
(Exit playmode first  
or add component manually)

Object-1

Object-2

Object-3



Refresh

Showing Frames: 8



[0]

☒ position

☒ rotation

☒ scale



☒ preview

You're in changes preview mode, click on a node and object in the scene will have its values adjusted to reflect its state at that time.

Frame: 264

Frame: 1

### Position

Originator AsmDefMover (Asm Def Ti Ⓞ)

Method Name SetPosition

Position

X

0

Y

5.5

Z

2

► Original Method Call

### Scale

Originator Mover02 (T

Method Name SetTransformV

Scale

X

0

Y

0

► Original Method Call

### Scale

Originator AsmDefMover (Asm Def Ti Ⓞ)

Method Name SetTransformValues

Scale

X

5

Y

2

Z

2

### Position

Originator AsmDefMover

Method Name SetPosition

Position

X

0

Y

3.5

## Assemblies

If your changes are still not captured make sure to select assemblies that could be making change. By default tool will only track user code.

Capture change  
from assembly?

(Total time taken 180ms, patched  
24 methods)

### ▼ User Code

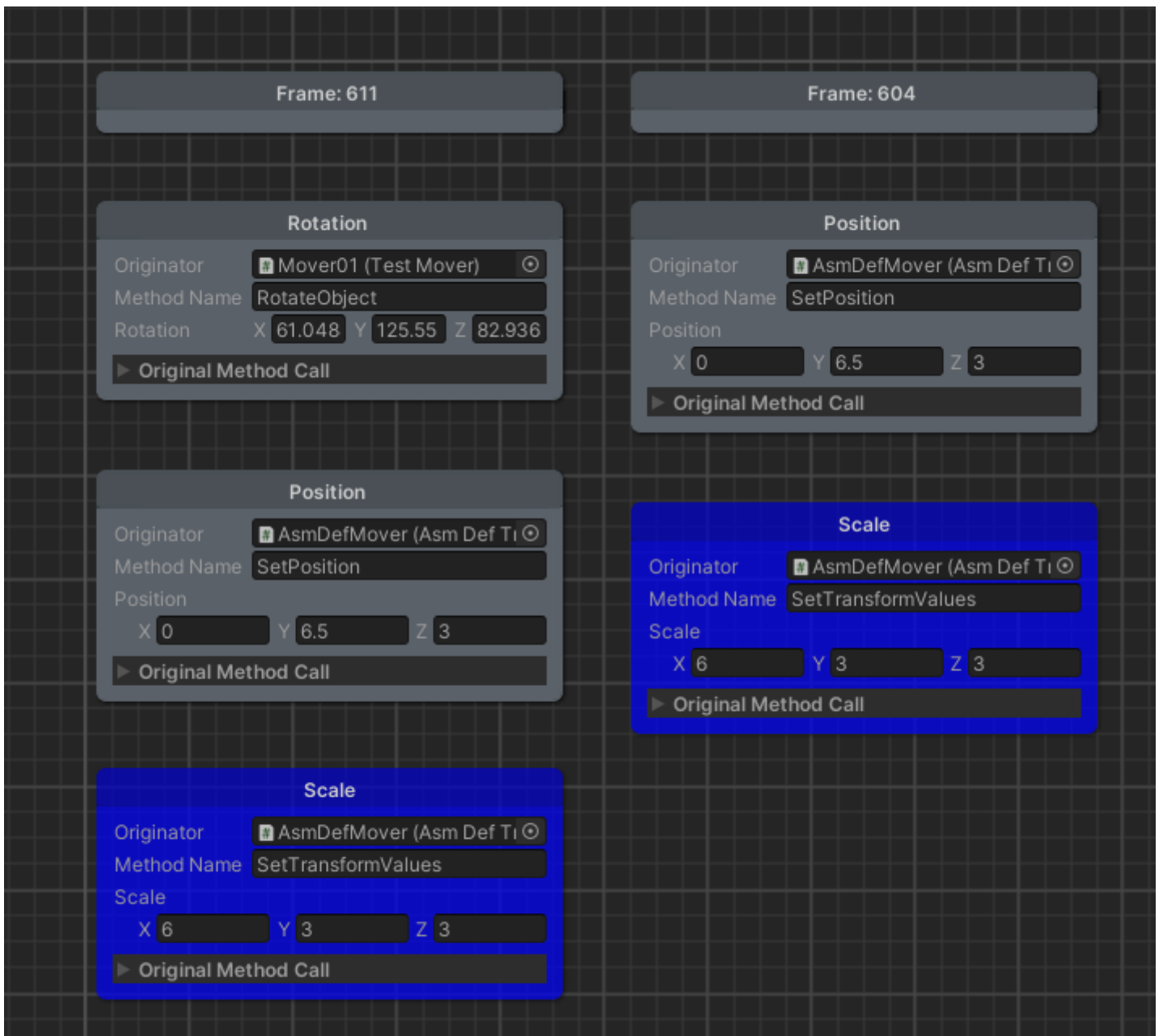
Assembly-CSharp.dll (patched 22 methods in 112ms) ☒

Assembly-CSharp-Editor.dll (not selected) ☐

Assembly-CSharp-Test.dll

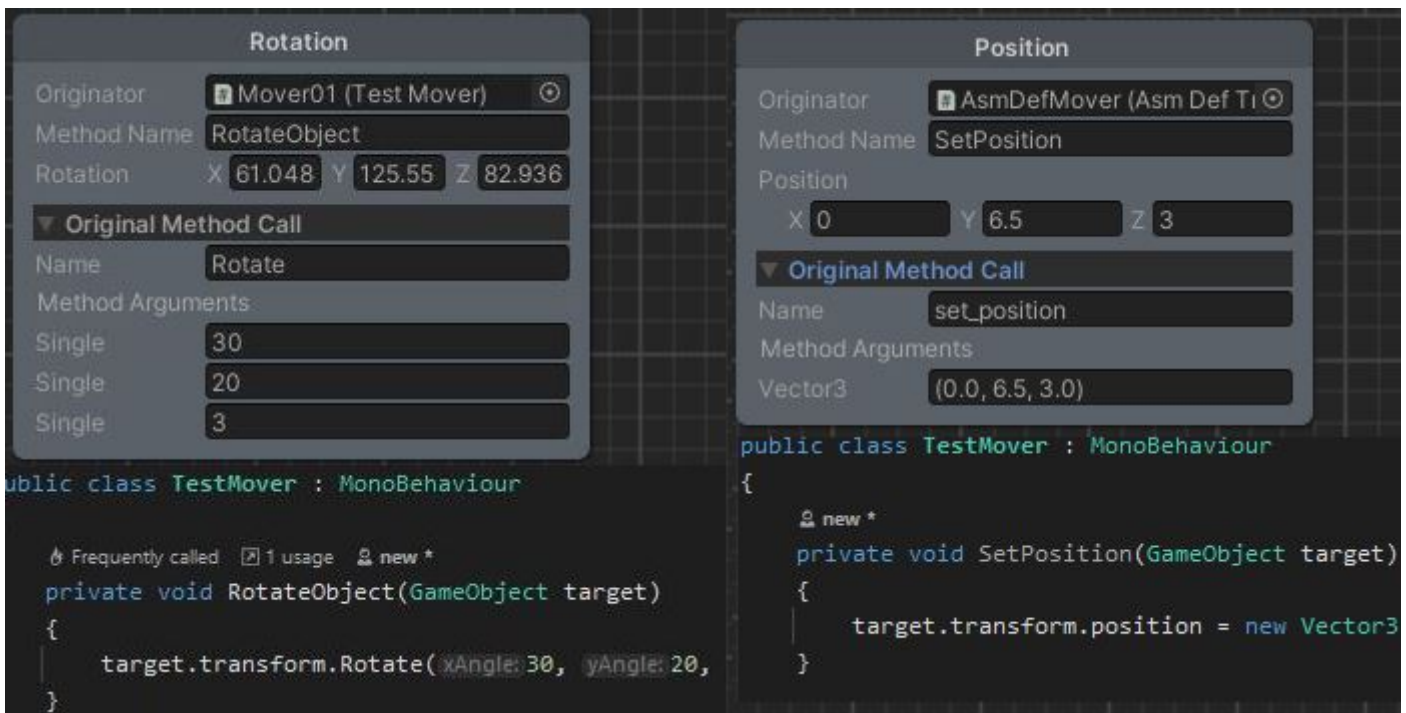
☐

## Main Changes Window



The main part of the tool will show changes for the selected object in a specified frame range. The first node indicates frame index that changes were captured in. Then specific changes made during that frame are listed vertically.

## Change Details



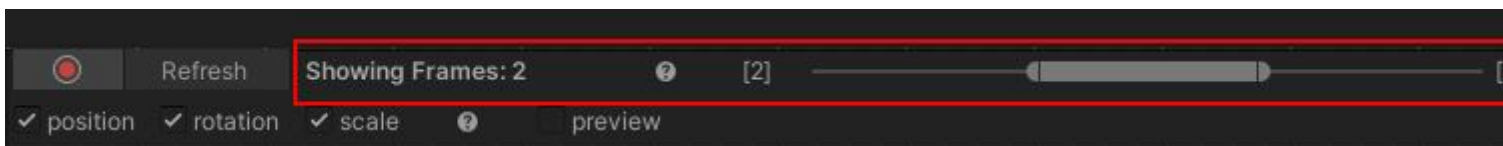
The single-node provides specific details about the change. - **Title** - this indicates whether a change has been made to position/rotation/scale - **Originator** - script instance that the change originated in. **[click to see it in Hierarchy View]** - **MethodName** - actual method in the script where the change originated. **[right click on the node title and choose 'goto code' to view the method in your code editor]** - **Rotation\Position\Scale** - (changes depending on node type) - value after the change has been made - **ValueBeforeChange** can be retrieved from code ([see API details](#)) - **Original Method Call** - actual method call that caused transform change - **Name** - the name for method, in case of setters this will be set\_XXX - **Method Arguments** - arguments passed to method

## Controlling the View - Navigation Bar

### Moving the View

- You can right-mouse-click (or use middle mouse-click) then drag to pan the view, that'll allow you to see different changes
- Zoom can be changed with a scroll wheel

### Frame Selection



You can choose which frames are visible using the top slider, currently visible frames are marked with numbers on left and right side.

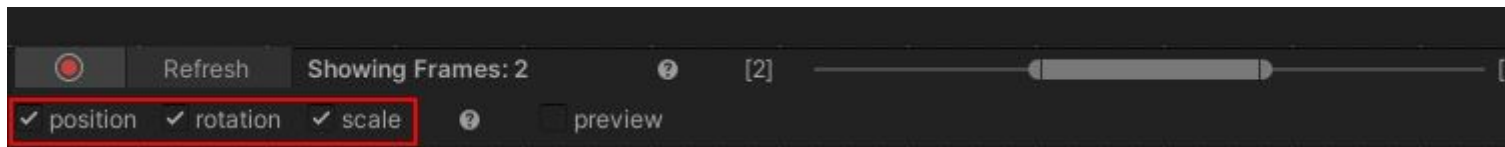
Eg. with setup like on screenshot above, tool will show changes: - starting at second ([2]) frame with captured data (start point [0] being the last frame with captured data) - ending at fourth ([4]) frame with captured data (start point [0] being the last frame with captured data)

You can adjust what's visible by: - dragging left bar - where data should start from - dragging right bar - where data should end at - dragging whole slider using middle part to change whole area that's visible

By default 1000 last frames change data is captured and you can select up to 100 frames to be rendered at a single time in the graph, those values can be changed via API if needed.

[\(MaxAllowedFramesToShowOnScreen / KeepChangesDataForMaximumNumberOfFrames\)](#)

## Change Type Visibility



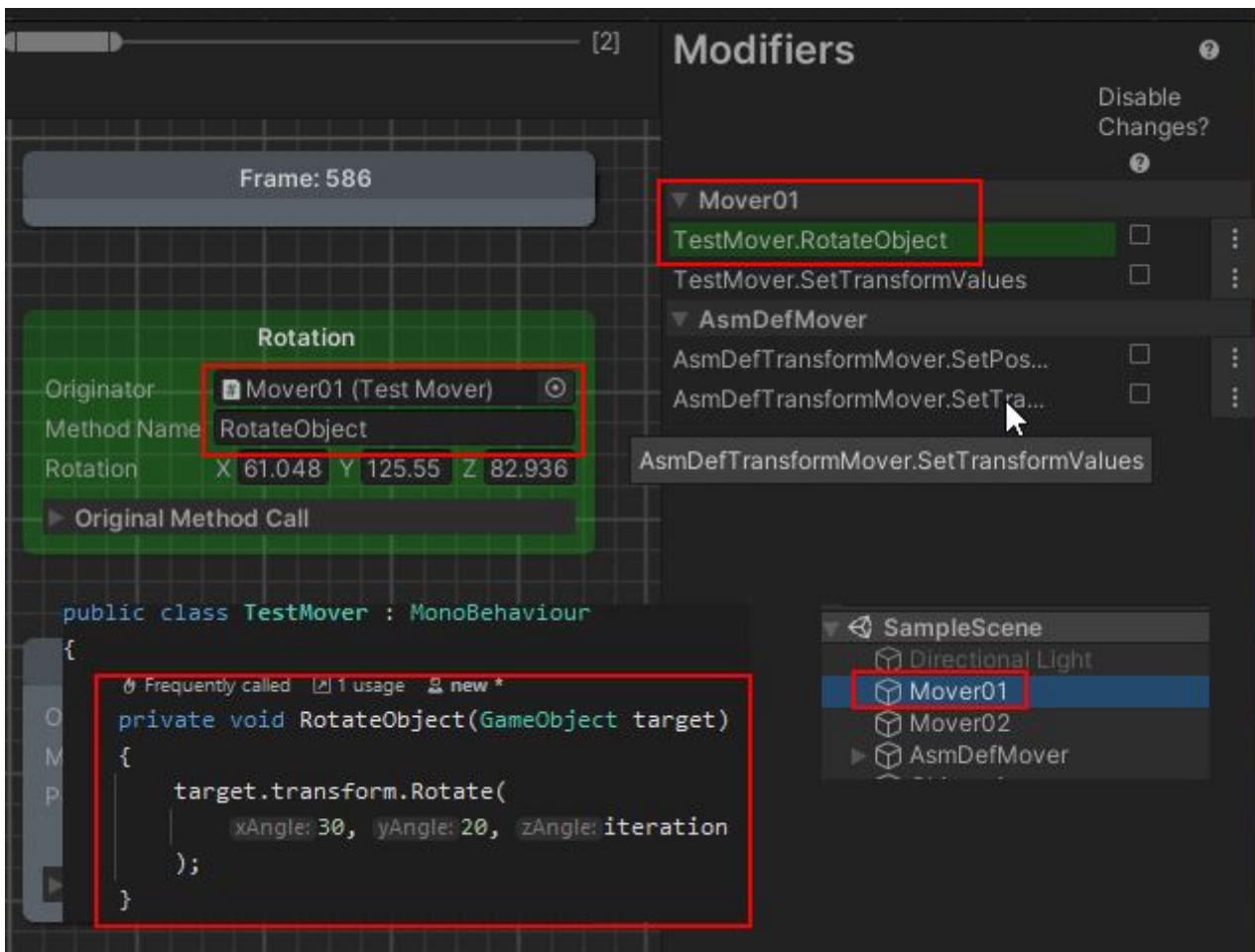
You can use checkboxes to adjust which specific change types you'd like to see. This can help you remove unnecessary nodes for example when you're looking at issues with position.

## Tracked Objects Sidebar



On the left side, you can see object names that are currently tracked. - clicking the buttons will show changes related to that object - currently selected object is indicated with green color - top button can be used to start tracking currently selected object (in Hierarchy View) [it'll add `TrackTransformChanges` to that object]

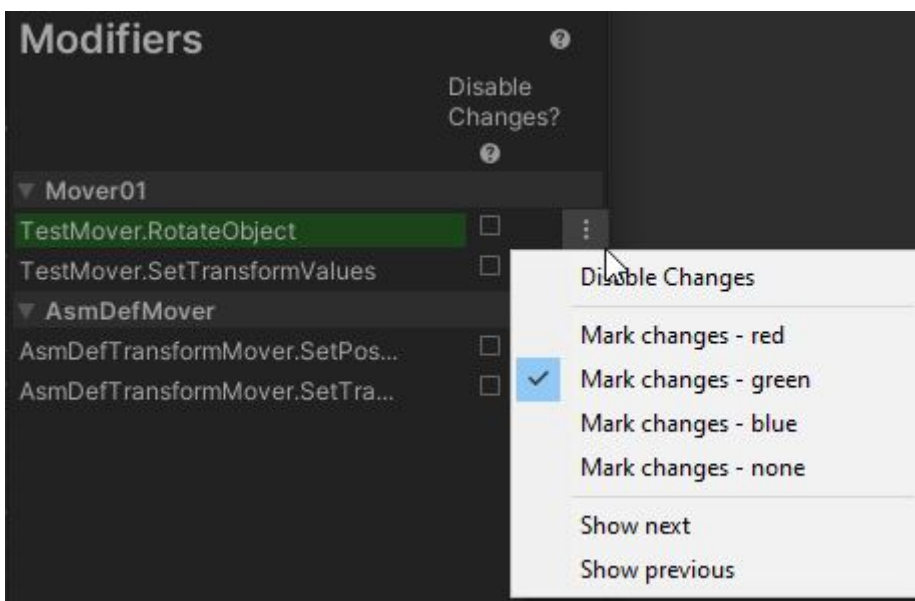
## Modifiers Sidebar



Changes visible on screen are grouped by TransformModifier - that is a specific change to object that originated from specific **game object instance** via **script** using **specific method**. This panel breaks them down to better illustrate which objects are changing tracked transform.

For example one of TransformModifiers on the screenshot is Mover01 game object that's modifying tracked object via TestMover script, in RotateObject method.

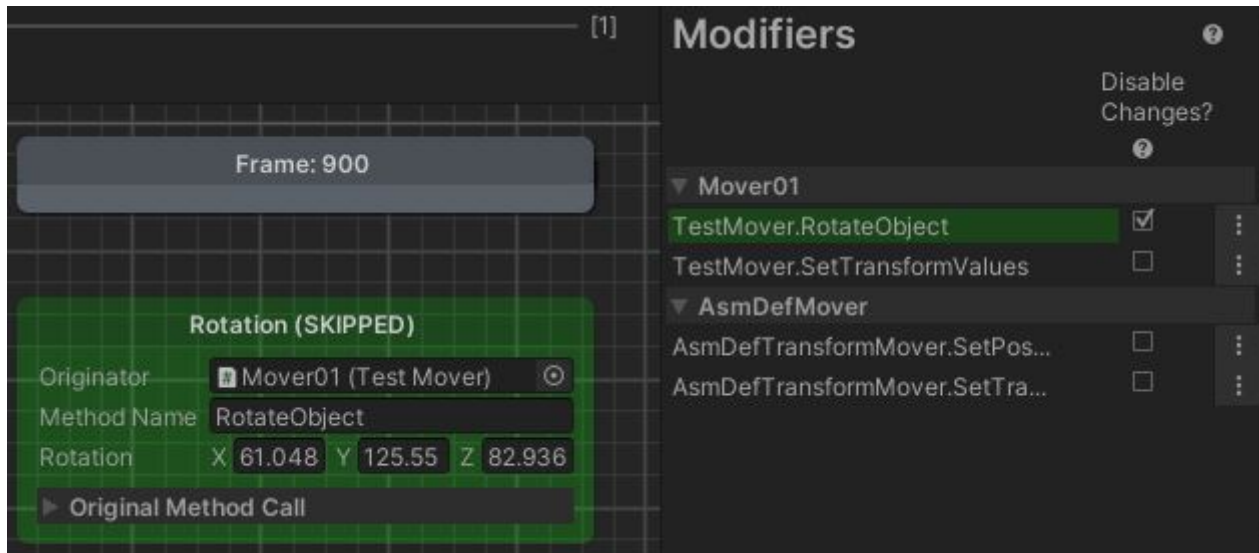
## Tracking Specific Modifiers



There are some per-modifiers actions available (accessed via ... button next to method name).

If you're interested in changes made by the modifier you can: - color code it - which makes it easier to find on the main graph - navigate to next/previous change made by it

## Temporarily Skipping Specific Changes at Runtime



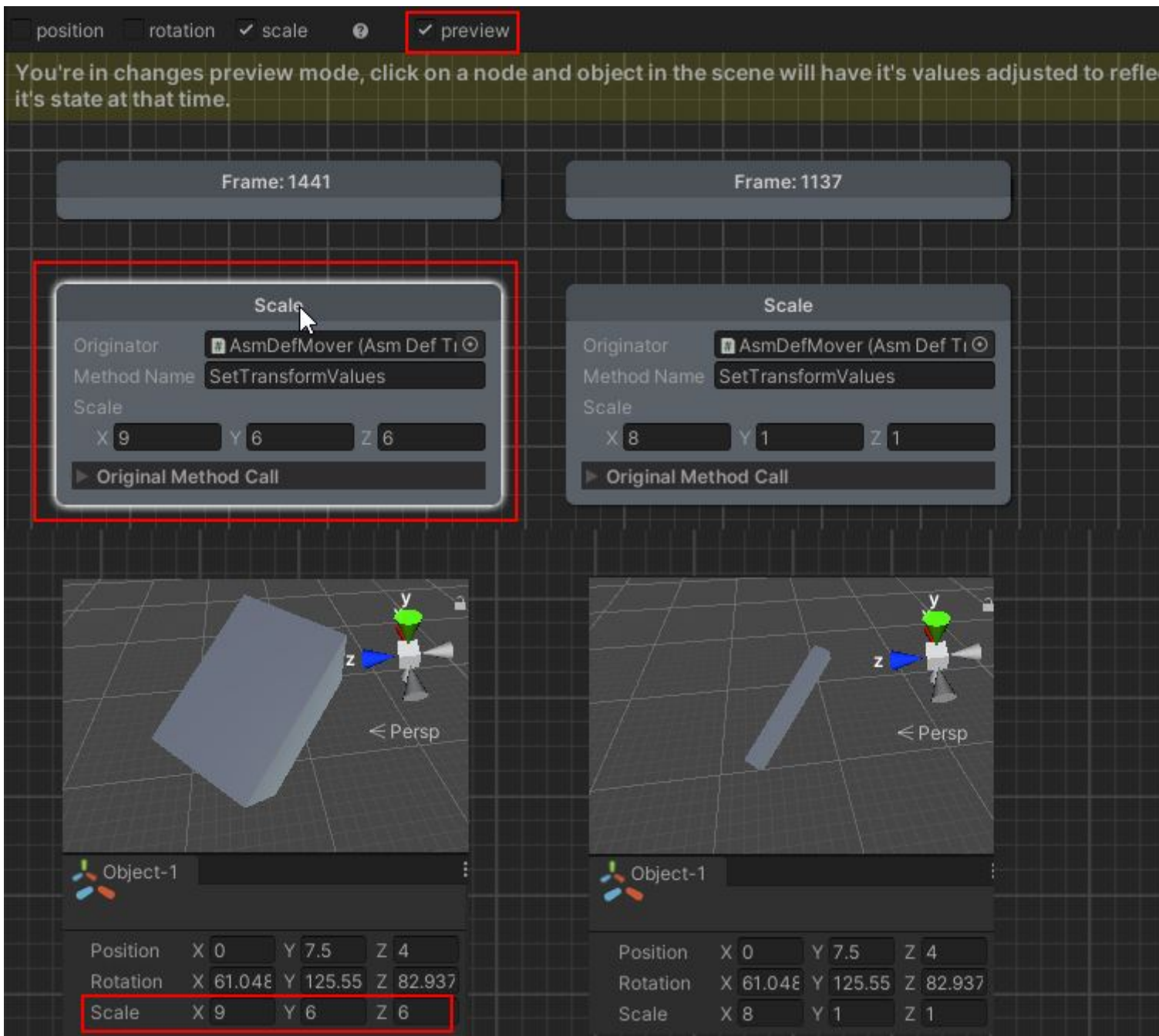
The tool will allow you to quickly isolate `TransformModifiers` that could be causing issues - this can be done by temporarily disabling changes made by them.

Simply tick the 'disable changes' checkbox next to the modifier and resume the game. **Any change made by modifier will now be skipped.** The actual change node will still be registered by the tool and the title will indicate that it was 'SKIPPED'

## Replay Previous Changes in Scene View

Sometimes it's not simple to find the issue by looking at raw values. You can easier pinpoint the issue by replaying specific changes directly in game/scene view. This will allow you to see the object the way it was at that change time. You can simply view the last few frames to see if the issue is visible there.

**To enable:** 1) tick 'preview' in the top bar 2) now you can click on a specific change node and change will be applied to the tracked object



## Capturing Changes - Configuring Sources / Assemblies Bar

## Assemblies

If your changes are still not captured make sure to select assemblies that could be making the change. By default tool will only

|                                         | (Patching will happen when entering playmode)<br>(Will take approximately 173ms, to patch 24 methods) | Capture change<br>from assembly     |
|-----------------------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------|
| ▼ User Code                             |                                                                                                       | <input type="checkbox"/>            |
| Assembly-CSharp.dll                     | (previously patched 22 methods in 109ms)                                                              | <input checked="" type="checkbox"/> |
| Assembly-CSharp-Editor.dll              | (not selected)                                                                                        | <input type="checkbox"/>            |
| ► Unity                                 |                                                                                                       | <input type="checkbox"/>            |
| ▼ Third Party                           |                                                                                                       | <input type="checkbox"/>            |
| AmplifyShaderEditor.dll                 | (not selected)                                                                                        | <input type="checkbox"/>            |
| AmplifyShaderEditor.Samples.BuiltIn.dll | (not selected)                                                                                        | <input type="checkbox"/>            |
| AsmDefTests.dll                         | (previously patched 2 methods in 64ms)                                                                | <input checked="" type="checkbox"/> |
| TransformChangesDebugger.Editor.dll     | (not selected)                                                                                        | <input type="checkbox"/>            |
| TransformChangesDebugger.Runtime.dll    | (not selected)                                                                                        | <input type="checkbox"/>            |

The tool will find all calls to any transform modifying methods (eg setters [`set_position`, `set_rotation`] but also more complex methods like `Transform.Rotate(1f, 230f, 15f, Space.Self)`). This happens directly in memory so no assembly files are actually modified. This process can add some time to entering play mode.

By default only changes originating from your own `User Code - Assembly-CSharp.dll` will be captured. Of course, this can not be enough if change originates from say 3rd party library.

You can use the bottom Assemblies bar to configure which assemblies should be included. Simply use the tick box next to the name.

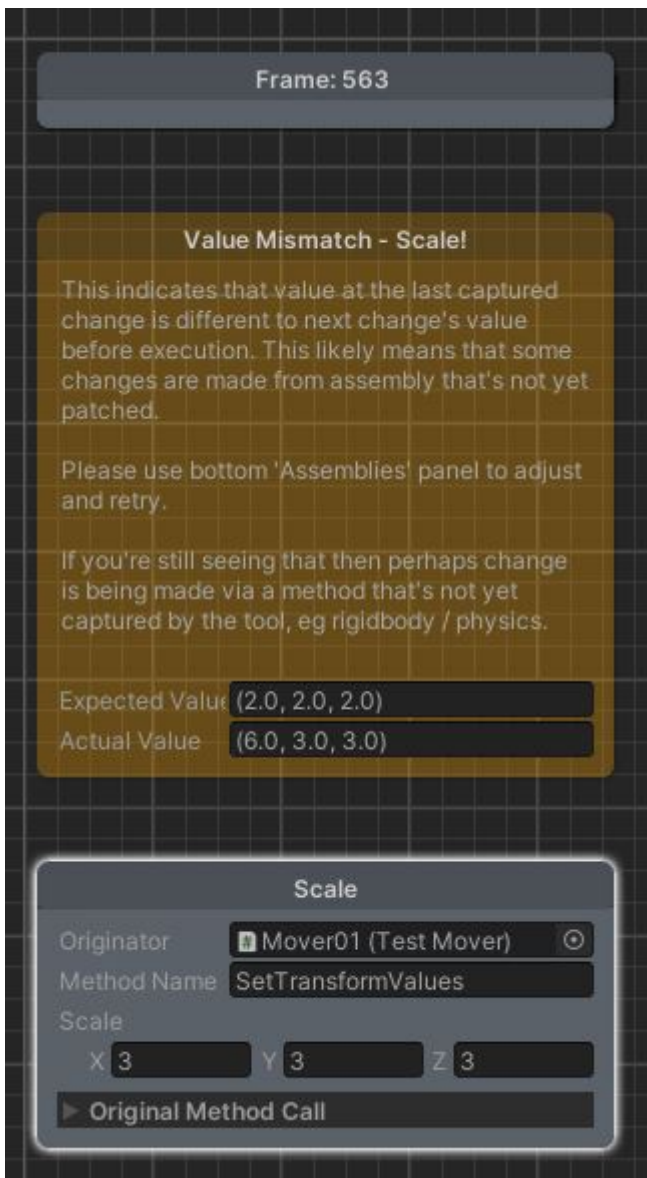
Assemblies are grouped into 3 main categories: - `User Code` - this is code in main assembly `Assembly-CSharp.dll` - `Unity` - all unity libraries that are loaded - `Third Party` - any code that's compiled into other assemblies. You don't need actual source code, the tool will also work with raw assemblies (DLL)

**You can also chose additional assemblies in play mode and they'll be patched on the fly.**

**It's easy to select all assemblies in the group by ticking checkbox next to it.**

## Change Mismatch

If some changes are being made to transform that tool did not register - it'll let you know. It does that by comparing last change value to next `ValueBeforeChange` - if there's any mismatch then it's likely some assemblies are not tracked.



## Physics

Currently physics (eg changes done via **Rigidbody** or gravity) will be treated as mismatch. If you're debugging transform that's misbehaving then it's best to temporarily turn gravity off. If physics is indeed the cause of the issue then this tool will not be able to help you at this point.

**Please let me know if that's important to you, while it's not officially supported I'll look into improving that part if there's enough interest.**

## Performance

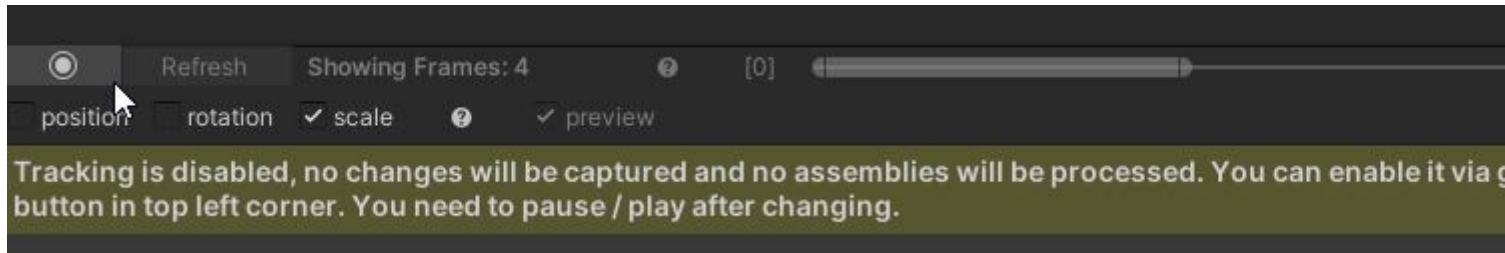
When you're processing assembly for the first time - tool needs to find all the methods that should be patched. That's most time-expensive part of the process but it only happens once (as long as assembly is not recompiled) - that means 3rd party / Unity libraries need to execute 'find' step just once to build cache. After that, search step is not executed and instead only patching happens - which is much quicker. If patched assembly is recompiled tool will also re-run search step and update cache.

You can see performance statistics around patching times to help you decide if specific library is needed.

This greatly improves workflow and helps keep entering play-mode time well under 0.5s.

**Assemblies will be only processed if there's at least 1 `TrackTransformChanges` script in the scene, you can have the tool imported into the project ready to be used without any performance cost if you're not using it.**

## Quickly Disable Tool to Save Time



You can very easily turn whole patching process off by turning the tool off with 'record' button in top bar.

## API Usage - Advanced

The tool exposes easy to use and documented API that you can use directly. Following section will document what's possible.

### Adding Custom Code to Handle Changes

You can execute custom code in response to any tracked change. The easiest way to do that is to use `UnityEvents` on `TrackTransformChanges`, `PositionChanged`, `RotationChanged` and `ScaleChanged`. They give you access to `TransformChange` which has change data ([API Reference](#))

You can also use

`TransformChangesDebuggerManager.RegisterCallbackFor[Position/Rotation/Scale]Changes` to register your custom callback. Eg. have a look at `ApiUsageExamples.cs`

```
TransformChangesDebuggerManager.RegisterCallbackForPositionChanges(InterceptFor
    (ilWeavedValues, newValue) =>
    {
        Debug.Log($"Intercepted scale change: {newValue} " +
            $"{ilWeavedValues.CallingFromMethodName} {ilWeavedValues.CallingFromType} " +
            $"{ilWeavedValues.ChangeType}");
    }
);
```

Will produce following result on changes: Intercepted scale change: (0.0, 0.0, 0.0) TestMover.SetTransformValues() Mover02 Scale Intercepted scale change: (1.0, 1.0, 1.0) TestMover.SetTransformValues() Mover02 Scale Intercepted scale change: (2.0, 2.0, 2.0) TestMover.SetTransformValues() Mover02 Scale

### Skipping Specific Changes

You can have more control over which changes are skipped (at runtime) by calling `TransformChangesDebuggerManager.SkipTransformChangesFor` and passing `ShouldSkipTransformChangeExecutionPredicate`.

For example following setup would skip any changes made to object named `DoNotChangeIfCallingFromSpecificMethod` when change is coming from method name `TestMover.SetPosition`.

```
TransformChangesDebuggerManager.SkipTransformChangesFor((ilWeavedValuesArray,
{
    if (changingComponent.name == "DoNotChangeIfCallingFromSpecificMethod" &&
    {
        return true; //true will indicate that change should be skipped
    }
    return false;
}));
```

To unregister you simply call `TransformChangesDebuggerManager.RemoveSkipTransformChangesFor` passing in the same predicate.

## Enable/Disable Tool

Change `TransformChangesDebuggerManager.IsTrackingEnabled`

## Controlling GUI

It's very easy to adjust any GUI properties programmatically, see [API Reference - TransformChangesDebuggerGuiManager](#) for details.

## Drawbacks / Limitations

### Debugging Original Code

When tool is enabled, calling code will be adjusted in memory that means you won't be able to hit breakpoints in some of affected code.

For example in following scenario debugger will not break if you set breakpoint anywhere in `SetPosition` method. `` private void SetPosition(GameObject target) { target.transform.position = new Vector3(iteration2, iteration +3.5f, iteration);

```
//you won't be able to add breakpoint to this method
var calculation = 1 + 2;

} ``
```

You'll however be able to hit breakpoints in calling code, eg `public void SetTransformValues() { foreach (var target in SetTargets) { SetPosition(target); } }`

This is due to the fact that methods that modify transform (be it via setters or via method calls) will have their IL instructions rewritten. In that case `SetTransformValues()` method will actually make a call to dynamically patched method `SetPosition_Patch0(target)` - with that change your breakpoint is simply never hit.

**You can very simply turn the tool off once you found the issue and breakpoints in affected code will work again.**

## Prod Build Support / Non Editor Usage

The core API that enables transform change tracking is fully independent of `UnityEditor`. You can use it in build and it'll work. `AdvancedAPIUsage` demo shows how easy that interception can be configured without using any parts of GUI.

However the tool development was focused on functionality rather than prod-required performance. You can find that in bigger solutions additional code executed affects performance (and additional IL code affects DLL size). This is to be expected. Currently I'd not recommend using it in production build.

**Please let me know if that's important to you, while it's not officially supported I'll look into improving that part if there's enough interest.**

## Concurrency

Currently changes will only work if transforms are adjusted on main/single thread.

**Please let me know if that's important to you, while it's not officially supported I'll look into improving that part if there's enough interest.**

## Method changing more than 1 property

Methods that are changing more than 1 property (position/rotation/scale) will be captured as change to position.

## No support for NET Standard 2.0

No support for NET Standard 2.0 API Compatibility level - currently you have to use .NET 4.x

For example: `Transform.SetPositionAndRotation()` - will be rendered as a position change, where rotation will be skipped entirely (you'll still get correct information about method and method arguments)

# API Reference

## TrackTransformChanges

Add this class to game objects that changes you want to be tracked.

- `PositionChanged` - Fired on every tracked position related change, could be used to pinpoint exact change that's causing issue in a programmatic way

- **RotationChanged** - Fired on every tracked rotation related change, could be used to pin point exact change that's causing issue in programmatic way
- **ScaleChanged** - Fired on every tracked scale related change, could be used to pin point exact change that's causing issue in programmatic way
- **HandlePositionChange** - Fired on every tracked position related change, override in derived class for custom handling
- **HandleRotationChange** - Fired on every tracked rotation related change, override in derived class for custom handling
- **HandleScaleChange** - Fired on every tracked scale related change, override in derived class for custom handling

## TransformChange

Details about specific change to transform.

- **CallingObject** - Object that initiated the change
- **CallingFromMethodName** - Method name where the change originated
- **TrackedDueToInterceptedMethodCallFullName** - Full method name that originally caused transform change, eg.  
`UnityEngine.Transform.set_position(UnityEngine.Vector3)` or  
`UnityEngine.Transform.Translate(UnityEngine.Vector3)`. Full list can be found in class `TranspiledMethodDefinitions`
- **TrackedDueToInterceptedMethodArguments** - Method arguments passed in to original changing method, eg for  
`UnityEngine.Transform.Rotate(UnityEngine.Vector3, UnityEngine.Space)`  
that could be { (0, 1, 2), `Space.Self` }
- **NewValue** - New value after change's been made
- **ValueBeforeChange** - Value right before change was made
- **ChangeType** - `ChangeType` - either position/rotation/scale
- **ChangedInFrame** - Frame index when change occurred
- **ModifiedObject** - Object which transform has been modified
- **WasChangeSkipped** - Indicates whether change was not made as it was skipped at runtime due to skip setup
- **PreviousSameTypeChange** - Previous tracked change of same type, this can be traversed to create a chain of changes of same type in reversed order
- **IsMismatchWithPreviousChange** - Indicates whether last change new value is different than current change's `ValueBeforeChange` which likely means there's a change that happened and was not captured

## TransformChangesDebuggerManager

Manager class that allows to control all backend aspects of transform change tracking, change skipping and assembly patching

- **IsTrackingEnabledChanged** - Fired when global change tracking was enabled/disabled
- **IsTrackingEnabled** - Controls change tracking for whole, if disabled no assemblies will be patched and no callbacks will be executed
- **Initialize** - Initializes change tracking by patching selected assemblies, call is only needed if you're not using default `TransformChangesDebuggerInitializer`
- **EnableChangeTracking** - Allows to enable change tracking for selected assemblies at runtime
- **RegisterCallbackForPositionChanges** - Allows to respond to position changes to specific object by executing handler

- **RegisterCallbackForScaleChanges** - Allows to respond to scale changes to specific object by executing handler
- **RegisterCallbackForRotationChanges** - Allows to respond to rotation changes to specific object by executing handler
- **RemoveCallback** - Removes previously created callback for changes to object
- **SkipTransformChangesFor** - Allows to skip specific changes at runtime for intercepted method calls. This is useful for eg. in debugging scenarios to narrow down which changes cause issues
- **SkipTransformChangesFor** - Allows to skip specific changes at runtime for intercepted method calls. This is useful for eg. in debugging scenarios to narrow down which changes cause issues
- **RemoveSkipTransformChangesFor** - Removed previously added predicate, effectively re-enabling changes
- **RemoveSkipTransformChangesFor** - Removed previously added modifier, effectively re-enabling changes

## TransformChangesDebuggerGuiManager

This class controls how the GUI behaves with various options

- **GraphDrawingDependentSettingChanged** - Fired when some significant change happened on main graph and signals to GUI it should reevaluate what's on screen
- **IsTrackingEnabled** - Controls tracking for whole tool, if disabled no assemblies will be patched. Convenience property - you should be setting `TransformChangesDebuggerManager.IsTrackingEnabled` instead
- **IsHistoryPreviewEnabled** - Controls if currently selected change is going to be applied to game object in pause mode (preview changes)
- **SelectedTrackedObject** - Controls which object changes are currently visible on main screen
- **SkipNFramesFromLastCaptured** - Controls how many frames should be skipped since last captured one to be displayed on screen. This works together with `ShowNFrames` to show changes from frame range
- **ShowNFrames** - Controls how many frames should be shown on screen. This works together with `ShowNFrames` to show changes from frame range
- **ShowChangesForPosition** - Controls whether position related changes should be visible on screen
- **ShowChangesForRotation** - Controls whether rotation related changes should be visible on screen
- **ShowChangesForScale** - Controls whether scale related changes should be visible on screen
- **AllTrackedObjects** - All objects that are currently tracked and can be selected in GUI
- **AssemblyPatchInfoEntries** - Information about assemblies that are available for patching or were patched. This also includes performance measurements
- **MaxAllowedFramesToShowOnScreen** - Limit amount of change nodes that can be shown on screen at given time - due to performance but also practical reasons this should be kept reasonable
- **NavigateToFirstNode** - Move node graph to first visible change node
- **NavigateToNextNode** - Move node graph to next node for same modifier
- **NavigateToPreviousNode** - Move node graph to previous node for same modifier
- **UseColorForChangesDoneViaModifiers** - Set background color for change nodes and modifiers to make changes of same type stand out better
- **StopUsingColorForChangesDoneViaModifiers** - Remove background color for change nodes and modifiers

## TransformChangesTracker

This class captures and makes sense of data captured for objects with `TrackTransformChanges` on a frame-by-frame basis

- `OnTransformChangeAdded` - Fires when new transform change has been captured for tracked object
- `KeepChangesDataForMaximumNumberOfFrames` - Maximum number of frames that data will be kept for. For performance reasons it's best to keep that within reasonable limits.
- `TrackChanges` - Starts tracking changes for specific object. This method needs to be called if subclassing
- `StopTrackingChanges` - Stops tracking changes for specific object
- `GetTrackedObjects` - Gets list of all tracked objects that there's any captured data for
- `GetTrackedObjects` - Gets list of all tracked objects that have tracking data for specific frame
- `GetFrameChangesForTrackedObject` - Resolves changes for specific frame and specific object
- `GetFrameChanges` - Resolves changes to all tracked object in specified frame
- `GetModifiers` - Gets all entries for tracked changes
- `GetModifiers` - Gets all entries for tracked changes in specific frame
- `GetModifiers` - Gets all entries for specific object
- `GetModifiers` - Gets all entries for tracked changes in frame